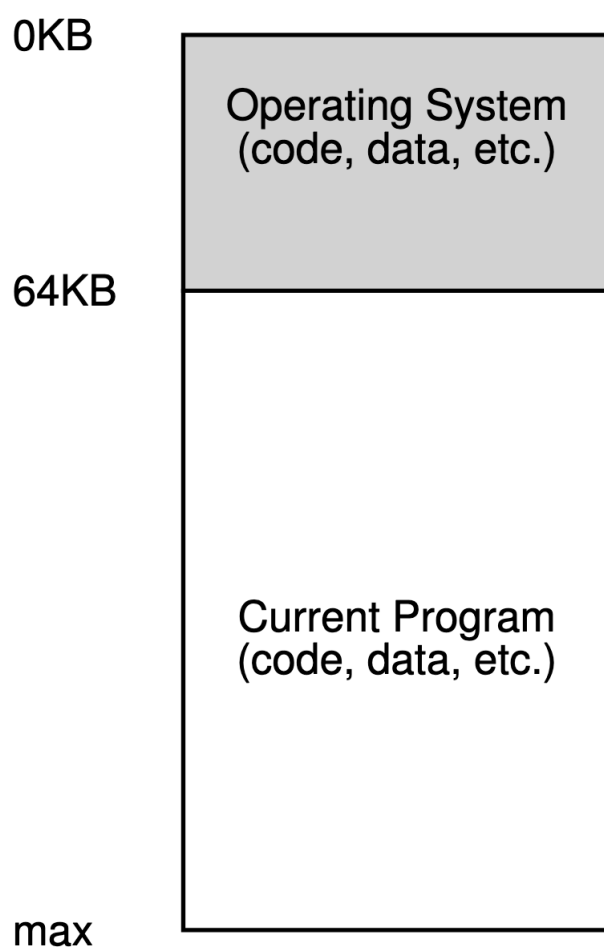


Early days:

The OS was a set of routines / a library that sat in the memory.

There was only one running program



Multi-programming:

Multiple processes are ready to run at a given time, and the OS would switch between them.

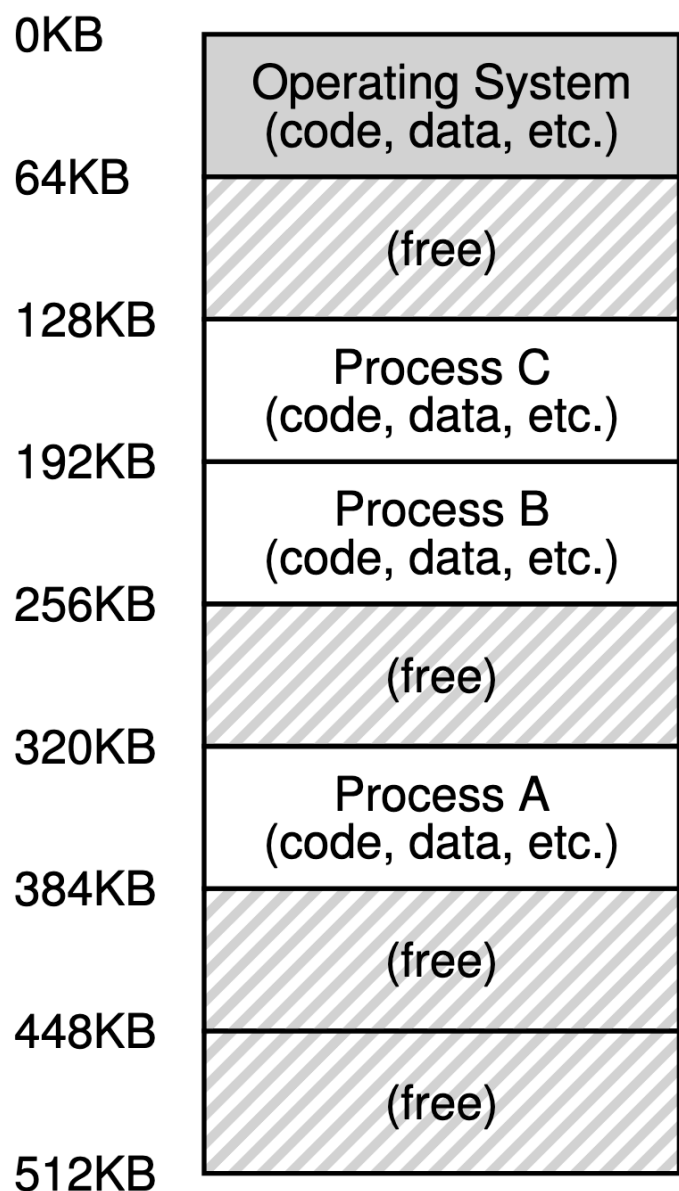
Many realized the limitations of batch computing.

Time sharing

One way to implement time sharing would be to run one process for a while giving it full access to all of memory, then stop it, save all of its state to some kind of disk, & load another process.

This is way too slow. Saving the entire contents of memory to disk is brutally non-performant.

So, we leave processes in memory while switching between them as below.



Process A, B, C each have a small part of memory carved out for them.

Now, we need to ensure processes don't read each other's memory.

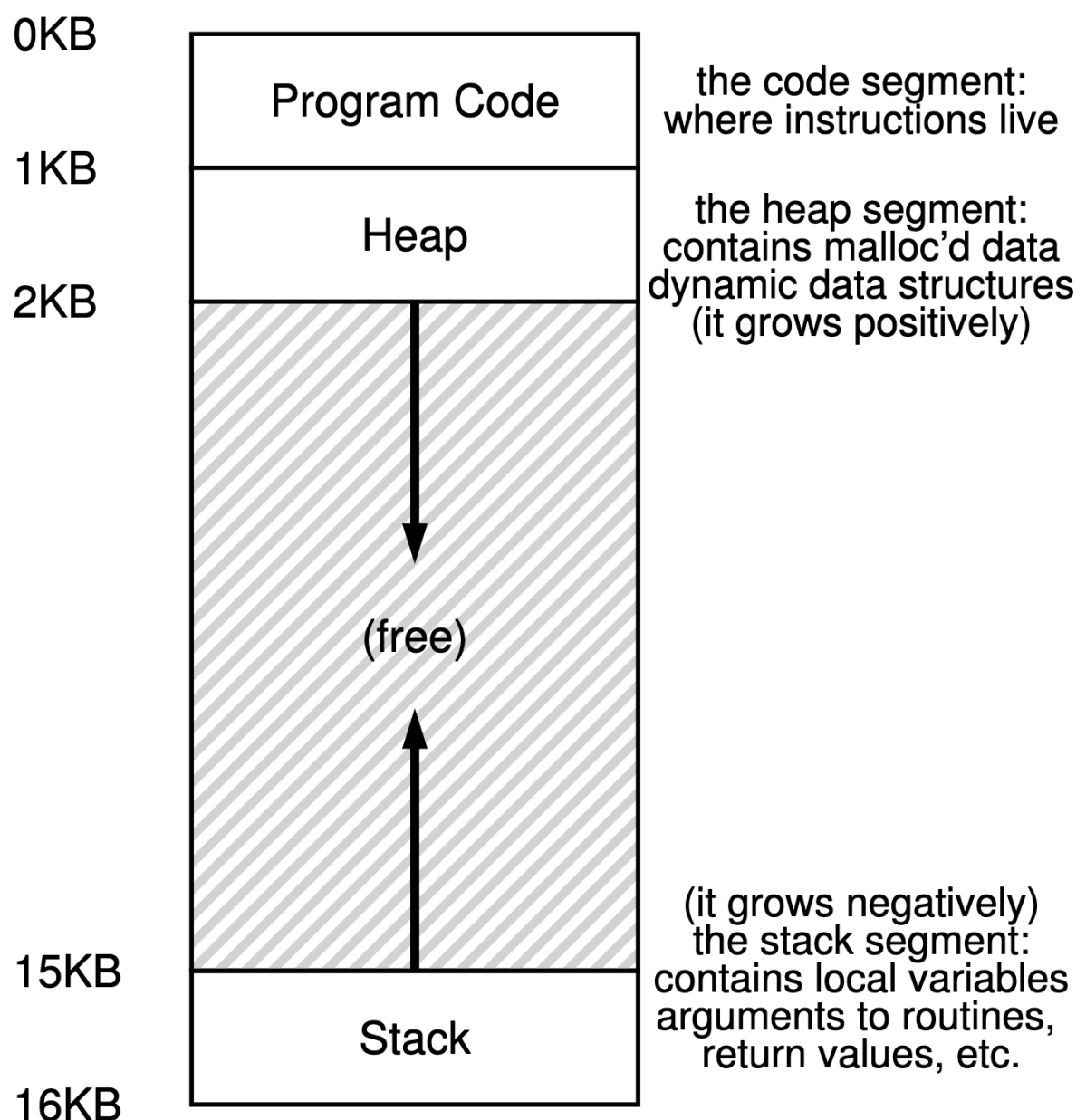
①

The Address Space:

It is the running program's view of memory in the system.

An address space of a process contains all the memory state of the running program.

both stack & heap want to be able to grow. This opposite arrangement allows this.



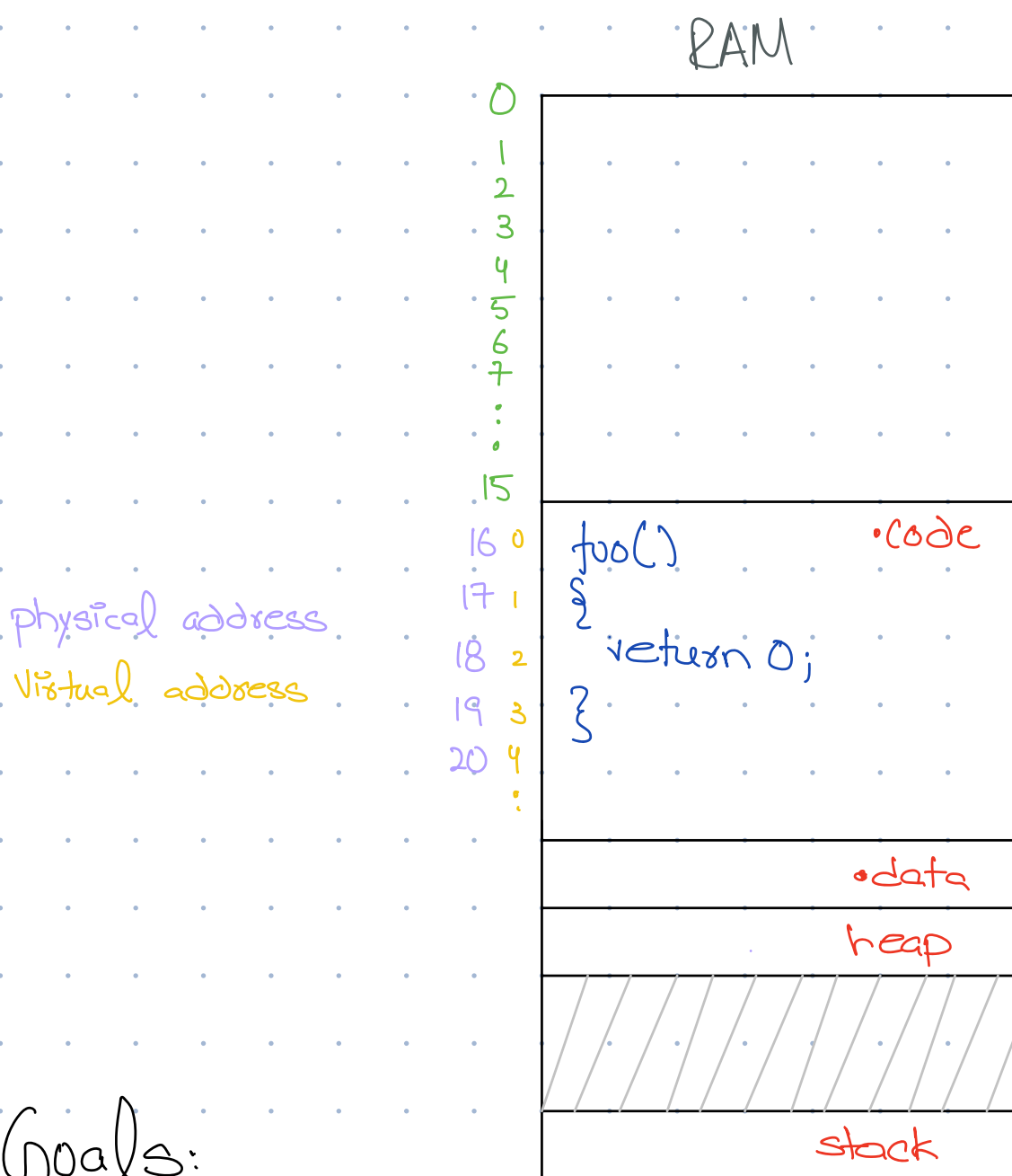
THE CRUX: HOW TO VIRTUALIZE MEMORY

How can the OS build this abstraction of a private, potentially large address space for multiple running processes (all sharing memory) on top of a single, physical memory?

Each program doesn't start at physical address 0, rather it is loaded in an arbitrary location ①.

A program thinks it starts at address 0 (Virtual address) but the OS knows the truth.

This is referred to as virtualization of memory.



Notes

↳ The OS with help from hardware will convert the virtual addresses into physical addresses.

You can think of them as offsets that can be added to physical addresses to get the actual address of the instruction in the program.

Goals:

1. Transparency:

↳ The OS should implement virtual in a way that is invisible to the running program.

Thus, a program should behave as if it has its own private physical memory.

2. Efficiency:

↳ The OS should make virtualization as efficient as possible both in terms of time (not making programs run much slower) and space (not using too much memory for structures needed to support virtualization).

3. Protection:

↳ The OS should make sure to protect processes from one another as well as the OS itself.

For instance,

When one program performs a load, a store, or an instruction fetch, it should not be able to access or affect in any way the memory contents of any other process or OS itself.

ASIDE: EVERY ADDRESS YOU SEE IS VIRTUAL

Ever write a C program that prints out a pointer? The value you see (some large number, often printed in hexadecimal), is a **virtual address**. Ever wonder where the code of your program is found? You can print that out too, and yes, if you can print it, it also is a virtual address. In fact, any address you can see as a programmer of a user-level program is a virtual address. It's only the OS, through its tricky techniques of virtualizing memory, that knows where in the physical memory of the machine these instructions and data values lie. So never forget: if you print out an address in a program, it's a virtual one, an illusion of how things are laid out in memory; only the OS (and the hardware) knows the real truth.

Here's a little program (`va.c`) that prints out the locations of the `main()` routine (where code lives), the value of a heap-allocated value returned from `malloc()`, and the location of an integer on the stack:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    printf("location of code : %p\n", main);
    printf("location of heap : %p\n", malloc(100e6));
    int x = 3;
    printf("location of stack: %p\n", &x);
    return x;
}
```

When run on a 64-bit Mac, we get the following output:

```
location of code : 0x1095afe50
location of heap : 0x1096008c0
location of stack: 0x7fff691aea64
```

From this, you can see that code comes first in the address space, then the heap, and the stack is all the way at the other end of this large virtual space. All of these addresses are virtual, and will be translated by the OS and hardware in order to fetch values from their true physical locations.